UNIVERSITY INSTITUTE OF ENGINEERING

Department of Computer Science & Engineering

(BE-CSE - 6th Sem)

Subject Name: System Design

Subject Code: 23CSH-314

SUBMITTED BY:

Name: Analava Bera
UID: 23BCS13611

Section: KRG-3A

SUBMITTED TO:

Er. Alok Kumar (E15012)

Q1. Explain the role of interfaces and enums in software design with proper examples.

Role of Interfaces

Interfaces define a contract that specifies **what a class must do** but not **how it does it**. They help achieve abstraction and improve flexibility and scalability in software systems.

Key Benefits of Interfaces

1. **Abstraction** – Hides implementation details.
2. **Loose Coupling** – Allows components to interact without depending on specific implementations.
3. **Code Reusability** – Multiple classes can implement the same interface.
4. **Scalability** – New implementations can be added without modifying existing code.

Example of Interface

```
interface PaymentProcessor {
   void processPayment(double amount);
}


class CreditCardPayment implements PaymentProcessor {
   public void processPayment(double amount) {
      System.out.println("Processing credit card payment: " + amount);
   }
}


class PayPalPayment implements PaymentProcessor {
   public void processPayment(double amount) {
```

```
        System.out.println("Processing PayPal payment: " + amount);
    }
}
```

Here, multiple payment methods implement the same interface, ensuring flexibility.

Role of Enums

Enums are special data types used to represent **a fixed set of constant values**. They improve readability, maintainability, and type safety.

Key Benefits of Enums

1. Prevents invalid values.
2. Improves code clarity.
3. Provides better compile-time checking.

Example of Enum

```
enum PaymentStatus {
    SUCCESS,
    FAILED,
    PENDING
}


class Payment {
    PaymentStatus status;
}
```

Here, PaymentStatus ensures that only valid statuses are used.

Q2. Discuss how interfaces enable loose coupling with example.

Loose Coupling

Loose coupling means reducing dependency between components so that changes in one component do not affect others.

Interfaces enable loose coupling by allowing systems to depend on **abstractions instead of concrete implementations**.

Example

Step 1: Define Interface

```
interface NotificationService {
   void sendNotification(String message);
}
```

Step 2: Implement Interface

```
class EmailNotification implements NotificationService {
   public void sendNotification(String message) {
      System.out.println("Email: " + message);
   }
}
class SMSNotification implements NotificationService {
   public void sendNotification(String message) {
      System.out.println("SMS: " + message);
   }
}
```

Step 3: Use Interface in Another Class

```
class OrderService {
   private NotificationService notificationService;
```

```
    public OrderService(NotificationService notificationService) {
        this.notificationService = notificationService;
    }
    public void placeOrder() {
        notificationService.sendNotification("Order placed successfully");
    }
}
```